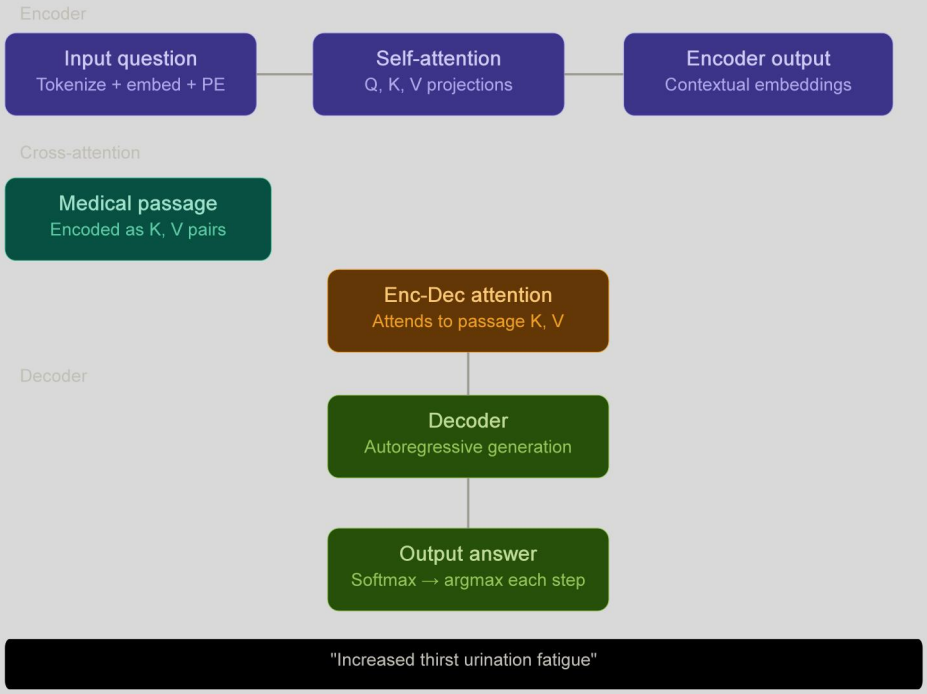
Deep Learning

Alaa Abozied 4221257

Transformar : https://github.com/alaa-abozied/transformar



1. Encoder Processing

How the question is encoded. The input `"What are the symptoms of diabetes?"` is first tokenized into subword tokens — roughly `["What", "are", "the", "symptoms", "of", "diabetes", "?"]`. Each token is mapped to a dense embedding vector of dimension `d_model` (e.g., 512), then a positional encoding vector is added element-wise so the model knows each token's position in the sequence. This combined vector is the input to the encoder stack.

Attention scores. The encoder computes three linear projections — Queries (Q), Keys (K), and Values (V) — from the embedding matrix X:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

The result is a seq_len × seq_len matrix where entry `[i, j]` represents how much token `i` should attend to token `j`. For our sentence, "symptoms" will have a high score toward "diabetes" since they're semantically linked, and "What" signals a question type that biases attention toward answer-bearing nouns.

Significance of self-attention. Without self-attention, each token is processed independently. Self-attention lets every token directly attend to every other token in a single layer — so "symptoms" immediately relates to "diabetes" without needing to pass through sequential hidden states like an RNN. This captures long-range dependencies in O(1) layers rather than O(n) steps, which is the core advantage over recurrent models.

2. Context Processing — Encoder-Decoder Attention

The decoder generates its Queries (Q) from its own hidden state. The encoder's representation of the medical passage produces Keys (K) and Values (V). Cross-attention computes:

CrossAttention(Q_decoder, K_passage, V_passage)

When the decoder is about to generate a symptom token, its query vector for "symptom-related concepts" will have the highest dot-product similarity with Keys corresponding to "increased thirst", "frequent urination", "extreme fatigue", and "blurred vision" — causing those Value vectors to dominate the attention-weighted sum. The model effectively retrieves the symptom list from the passage to condition each generation step.

---

3. Decoder Prediction — Greedy Decoding

At each step, the decoder samples the token with the highest softmax probability (greedy decoding / argmax):

Step	Selected token	Probability
1	Increased	0.75
2	thirst	0.80
3	urination	0.60
4	fatigue	0.70

Final generated answer: `"Increased thirst, urination, fatigue"`

Note that "frequent" and "extreme" were available candidates at steps 3 and 4 (0.3 and 0.2 respectively) but lost to "urination" and "fatigue" under greedy decoding. A beam search would explore multiple hypotheses simultaneously and may have recovered the more fluent `"Increased thirst, frequent urination, extreme fatigue"`.

```
PS D:\Downloads>  & 'C:\Users\HELAL\AppData\Local\Programs\Python\Python311\python.exe'
libs\debugpy\launcher' '60814' '--' 'd:\Downloads\self_attention.py'
    symptoms      0.1698      0.1659      0.1621      0.1643      0.1676      0.1702
          of      0.1692      0.1666      0.1644      0.1649      0.1662      0.1687
    diabetes      0.1698      0.1681      0.1645      0.1642      0.1651      0.1682

Encoder output shape : (6, 8)  (seq_len=6, d_k=8)
Row sums (must be 1) : [1. 1. 1. 1. 1. 1.]
PS D:\Downloads>
```

self_attention.py 2

D: > Downloads > self_attention.py > print_attention_matrix

```python
import numpy as np


class SelfAttention:
    def __init__(self, d_model: int, d_k: int):
        self.d_k = d_k
        rng = np.random.default_rng(42)
        self.W_q = rng.normal(0, 0.1, (d_model, d_k))
        self.W_k = rng.normal(0, 0.1, (d_model, d_k))
        self.W_v = rng.normal(0, 0.1, (d_model, d_k))

    def forward(self, X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        Q = X @ self.W_q
        K = X @ self.W_k
        V = X @ self.W_v
        raw_scores = Q @ K.T / np.sqrt(self.d_k)
        weights = self._softmax(raw_scores)
        return weights @ V, weights

    @staticmethod
    def _softmax(x: np.ndarray) -> np.ndarray:
        e = np.exp(x - x.max(axis=-1, keepdims=True))
        return e / e.sum(axis=-1, keepdims=True)


class TransformerEncoder:
    def __init__(self, vocab_size: int = 10_000, d_model: int = 16, d_k: int = 8):
        self.d_model = d_model
        rng = np.random.default_rng(0)
        self.embedding_matrix = rng.normal(0, 0.1, (vocab_size, d_model))
        self.attention = SelfAttention(d_model, d_k)

    def _positional_encoding(self, seq_len: int) -> np.ndarray:
        positions = np.arange(seq_len)[:, None]
        dims = np.arange(0, self.d_model, 2)[None, :]
        angles = positions / (10_000 ** (dims / self.d_model))
        PE = np.zeros((seq_len, self.d_model))
```

D: > Downloads > self_attention.py > print_attention_matrix

```python
class TransformerEncoder:
    def _positional_encoding(self, seq_len: int) -> np.ndarray:
        PE[:, 0::2] = np.sin(angles)
        PE[:, 1::2] = np.cos(angles[:, : PE[:, 1::2].shape[1]])
        return PE

    def encode(self, tokens: list[str]) -> tuple[np.ndarray, np.ndarray]:
        indices = [hash(t) % self.embedding_matrix.shape[0] for t in tokens]
        X = self.embedding_matrix[indices] + self._positional_encoding(len(tokens))
        return self.attention.forward(X)


def print_attention_matrix(weights: np.ndarray, tokens: list[str]) -> None:
    col_w = 12
    header = f"{'':>{col_w}}" + "".join(f"{t:>{col_w}}" for t in tokens)
    print(header)
    print("-" * col_w * (len(tokens) + 1))
    for i, token in enumerate(tokens):
        row = f"{token:>{col_w}}" + "".join(f"{weights[i, j]:>{col_w}.4f}" for j in range(len(tokens)))
        print(row)


if __name__ == "__main__":
    sentence = "What are the symptoms of diabetes"
    tokens = sentence.lower().split()

    encoder = TransformerEncoder(d_model=16, d_k=8)
    output, attention_weights = encoder.encode(tokens)

    print(f'Sentence : "{sentence}"')
    print(f"Tokens   : {tokens}\n")
    print("Self-Attention Score Matrix (softmax weights):")
    print_attention_matrix(attention_weights, tokens)
    print(f"\nEncoder output shape : {output.shape}  (seq_len={len(tokens)}, d_k=8)")
    print(f"Row sums (must be 1) : {attention_weights.sum(axis=1).round(6)}")
```